

Homework #2 Solutions Spring 2026

2026-03-06

Problem #2

Problem statement

We discussed K -fold cross validation in class, including the special case of Leave One Out Cross Validation (LOOCV). The link below gives a good discussion of the question of the optimal value of K ,

<https://stats.stackexchange.com/questions/61783/bias-and-variance-in-leave-one-out-vs-k-fold-cross-validation>

Using this link as a guide, conduct a small simulation study of the question of what the optimal K is for K -fold cross validation. The central question is that if bias and variance in LOOCV versus other values of K and whether LOOCV is the best method. This is open ended, but you should produce figures similar to those at the link. You should do this in R. Note that there is Python code provided, which should be helpful as a guide.

Introduction

We are interested in determining the optimal value of K in K -fold cross-validation. The goal of the cross-validation is estimate the expected prediction error when we apply the model to a new data set. Since we don't (typically) have new data sets, we use cross-validation.

There are two competing concerns when choosing K :

1. Fitting the models with higher sample size will result in better model fits. The higher the value of K , the higher the sample size used in the model fits.
2. The higher K is, the more similar the data sets and therefore the more correlated the results from the cross-validation. This may lead to higher variance.

To understand the second point, consider a situation where the data set happens to have a bias. For example, by chance, there are a large number of data points where high values of a particular predictor are paired with high values of the response. This will result in a higher estimated regression coefficient for that predictor, but this is in error. If we were estimating prediction error with independent data sets, then this same bias wouldn't exist in (most of) the test sets and so the estimated prediction error will be correctly higher. If we are doing LOOCV, then all of the test data sets will have the same bias and so the estimated prediction error will be too low. If we did e.g. 5-fold cross-validation, then we would probably get some test data sets that don't have the same bias and so the prediction error will be estimated more accurately.

This is the reasoning behind the idea that higher K may be sub-optimal. However, this reasoning may not be correct in practice and thus we do simulations.

Simulation Outline

Following the link, we make a simulation as follows:

Loop over K :

{ Loop over iterations 1 to R :

```

| { 1. Generate  $N$  data points from  $\sin(x) + \epsilon$ .
| 2. Perform  $K$ -fold cross-validation.
| 3. Store the MSE for the current  $K$  and current iteration  $i$ .
| }
| 4. Calculate the mean and variance of the MSE over  $i$  for the current  $K$ .
| }
Plot mean MSE versus  $K$ .

```

Simulating the data

First we will make a plot of the simulated data. This is to make sure that our simulation parameters are reasonable. Following the simulation at the link, I will generate x values from a uniform distribution over some range (the range doesn't matter) and y values as

$$y = \sin(x) + \epsilon$$

The values of ϵ are generated from a uniform distribution:

$$\epsilon \sim \text{uniform}(\min = -s, \max = s)$$

That is, the error is distributed uniformly about zero.

Next, I will plot the simulated data as a reality check.

First, I will do it with low variance so that I can clearly see the shape of the sine curve:

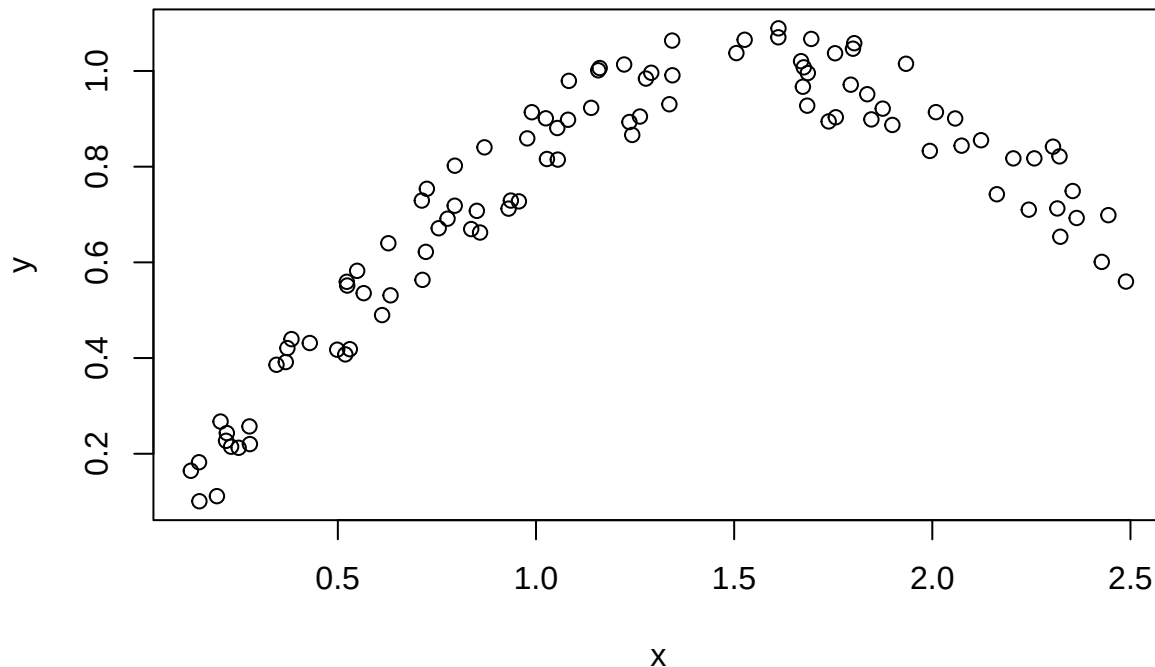
```

N=100
s=0.1

x=runif(N,min=0,max=2.5)
y=sin(x)+runif(N,min=-s,max=s)

plot(x,y)

```

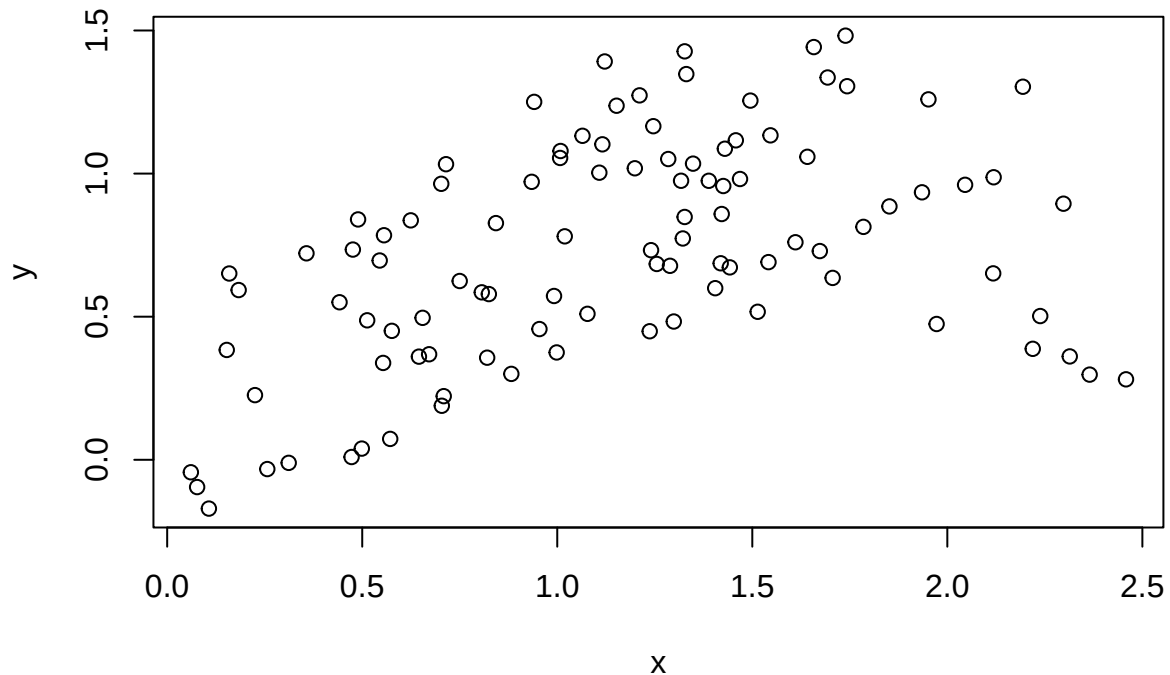


Now I will repeat with higher variance as at the link:

```
N=100
s=0.5

x=runif(N,min=0,max=2.5) # generate x values
y=sin(x)+runif(N,min=-s,max=s) #generate y values

plot(x,y)
```



This seems reasonable to start with.

Fitting the data

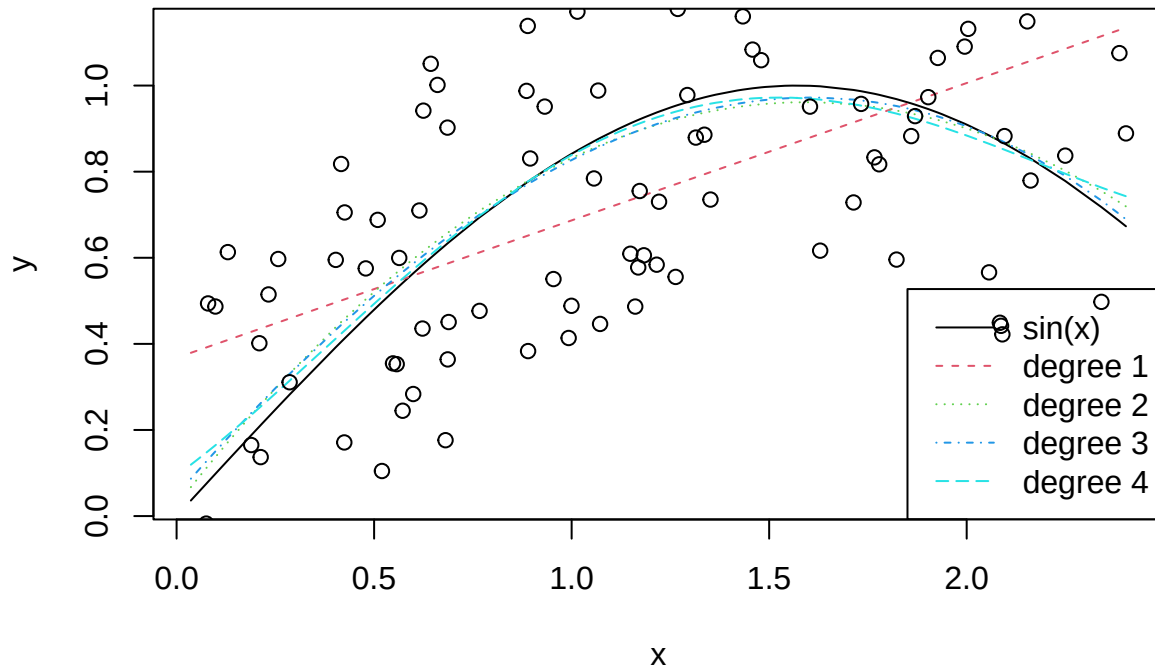
Next, I would like to see how well we can fit the data with different degrees of polynomial. The figure below shows the true curve ($\sin(x)$) along with the fitted 1st-4th degree polynomials.

```
set.seed(10)
N=100
s=0.5

x=sort(runif(N,min=0,max=2.5)) #generate x values
y=sin(x)+runif(N,min=-s,max=s) #generate y values

#put in a data frame and fit 1st to 4th degree polynomials:
dat=data.frame(x,y)
glm.fit1=glm(y~x,data=dat)
glm.fit2=glm(y~x+I(x^2),data=dat)
glm.fit3=glm(y~x+I(x^2)+I(x^3),data=dat)
glm.fit4=glm(y~x+I(x^2)+I(x^3)+I(x^4),data=dat)
```

```
matplot(x,cbind(sin(x),fitted(glm.fit1),fitted(glm.fit2),fitted(glm.fit3),fitted(glm.fit4)),type="l",col=1:5)
legend("bottomright",c("sin(x)","degree 1","degree 2","degree 3","degree 4"),lty=1:5,col=1:5)
points(x,y)
```



```
#plot(x,fitted(glm.fit2))
```

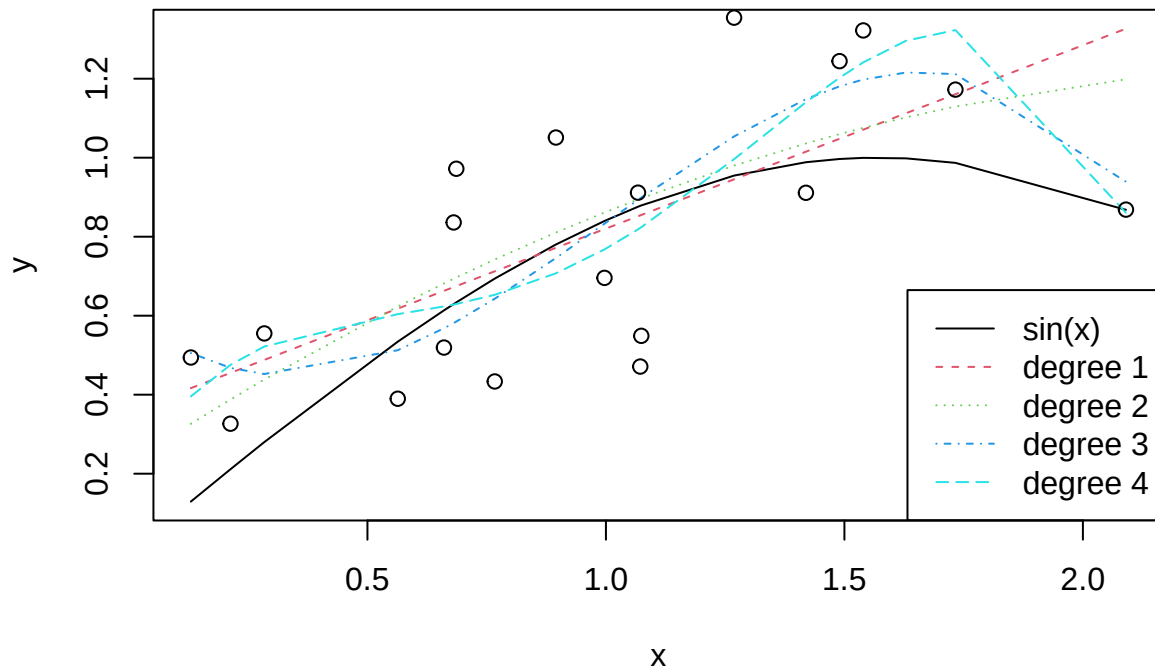
The first degree polynomial (straight line) fits poorly. The 2nd,3rd, and 4th degree polynomials all fit very well and there is negligible difference between them. Here is another example with lower sample size:

```
set.seed(10)
N=20
s=0.5

x=sort(runif(N,min=0,max=2.5)) #generate x values
y=sin(x)+runif(N,min=-s,max=s) #generate y values

#put in a data frame and fit 1st to 4th degree polynomials:
dat=data.frame(x,y)
glm.fit1=glm(y~x,data=dat)
glm.fit2=glm(y~x+I(x^2),data=dat)
glm.fit3=glm(y~x+I(x^2)+I(x^3),data=dat)
glm.fit4=glm(y~x+I(x^2)+I(x^3)+I(x^4),data=dat)

matplot(x,cbind(sin(x),fitted(glm.fit1),fitted(glm.fit2),fitted(glm.fit3),fitted(glm.fit4)),type="l",col=1:5)
legend("bottomright",c("sin(x)","degree 1","degree 2","degree 3","degree 4"),lty=1:5,col=1:5)
points(x,y)
```



```
#plot(x,fitted(glm.fit2))
```

All of the models do less well and there is not a clear difference in which degree polynomial does the best. I will follow the link and use a 4th degree polynomial.

Adding in simulation replicates

Next, we will make the loop over simulation replicates with K fixed. We generate the data, fit the model, conduct k -fold cross-validation, and save the estimated prediction error. Note that the simulation at the link generates a data set of size 10,000 and then samples from that for each simulation replicate. I generate the data for each replicate from the original distribution. This is simpler to code and I don't see a reason to do the more complicated way,

```
library(boot)
R=1000
N=100
sigma0=0.5
Kval=10
cv.error=vector(length=R) #vector to hold estimates of MSE for each replicate

#loop over replicates
for(i in 1:R)
{ x=runif(N,min=0,max=2.5)
  y=sin(x)+rnorm(N,sd=sigma0)
  dat=data.frame(x,y)
  glm.fit=glm(y~x+I(x^2)+I(x^3)+I(x^4),data=dat)

  cv.error[i]=cv.glm(dat,glm.fit,K=Kval)$delta[1] #conduct k-fold CV and store MSE
}
```

```
mean(cv.error)
```

```
## [1] 0.2660973
```

```
sd(cv.error)
```

```
## [1] 0.04121487
```

`mean(cv.error)` and `sd(cv.error)` give the mean and standard deviation of the MSE across simulation replicates. These are our main outputs. The prediction error will vary between different test data sets. Some data sets will be more representative of the underlying data generating process than others and thus they will produce better models. The mean gives an overall measure of the prediction error and the standard deviation shows how much we can expect the prediction error to vary between data sets.

Adding the loop over K

Finally, we add a loop over K produce our target output: a plot of $1 - \text{mean}(MSE)$ and $\text{sd}(MSE)$.

Run the code for a sample size of 40:

```
set.seed(12345)
R=1000
N=40
s=0.5
Kvals40=c(2:4,seq(from=5,to=N,by=5)) #range of K values to test
meanvals40=vector(length=length(Kvals40))
sdvals40=vector(length=length(Kvals40))
```

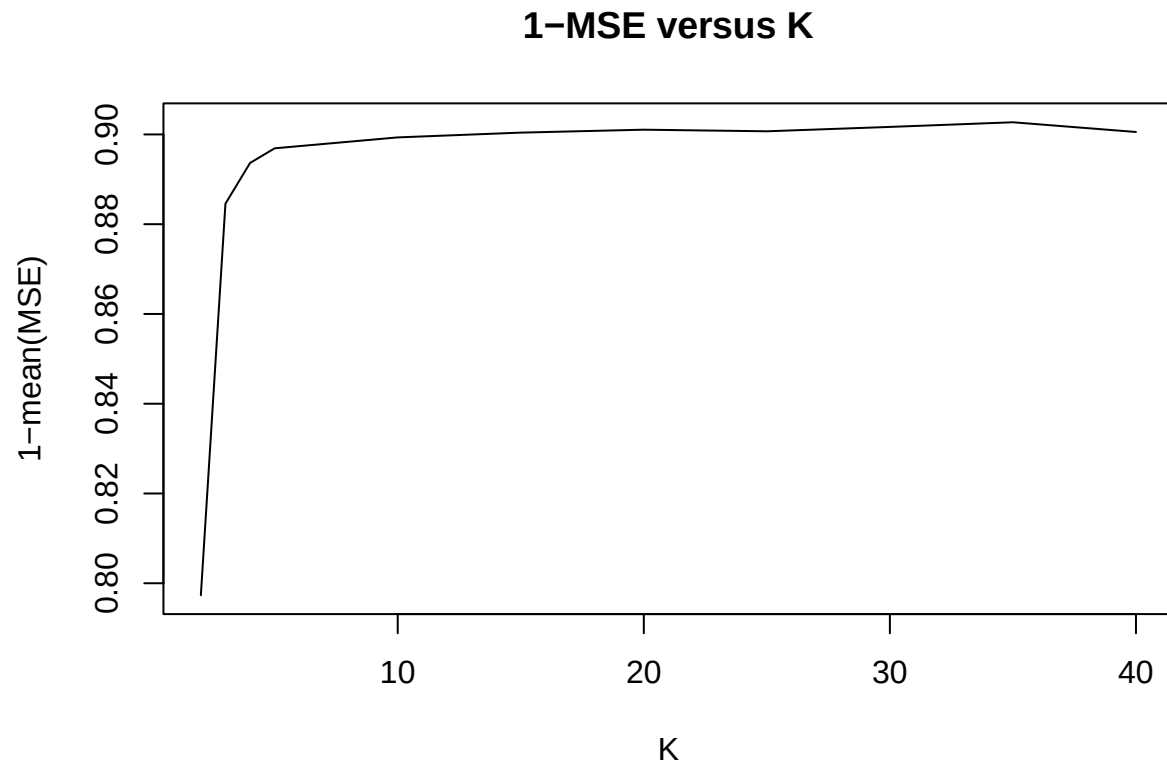
```
j=0
for(K in Kvals)
{ j=j+1
  print(K)
  cv.error=vector(length=R)

  for(i in 1:R)
  { x=runif(N,min=0,max=2.5)
    y=sin(x)+runif(N,min=-s,max=s) #rnorm(N,sd=sigma0)
    dat=data.frame(x,y)
    glm.fit=glm(y~x+I(x^2)+I(x^3)+I(x^4),data=dat)
    cv.error[i]=cv.glm(dat,glm.fit,K=K)$delta[1]
  }

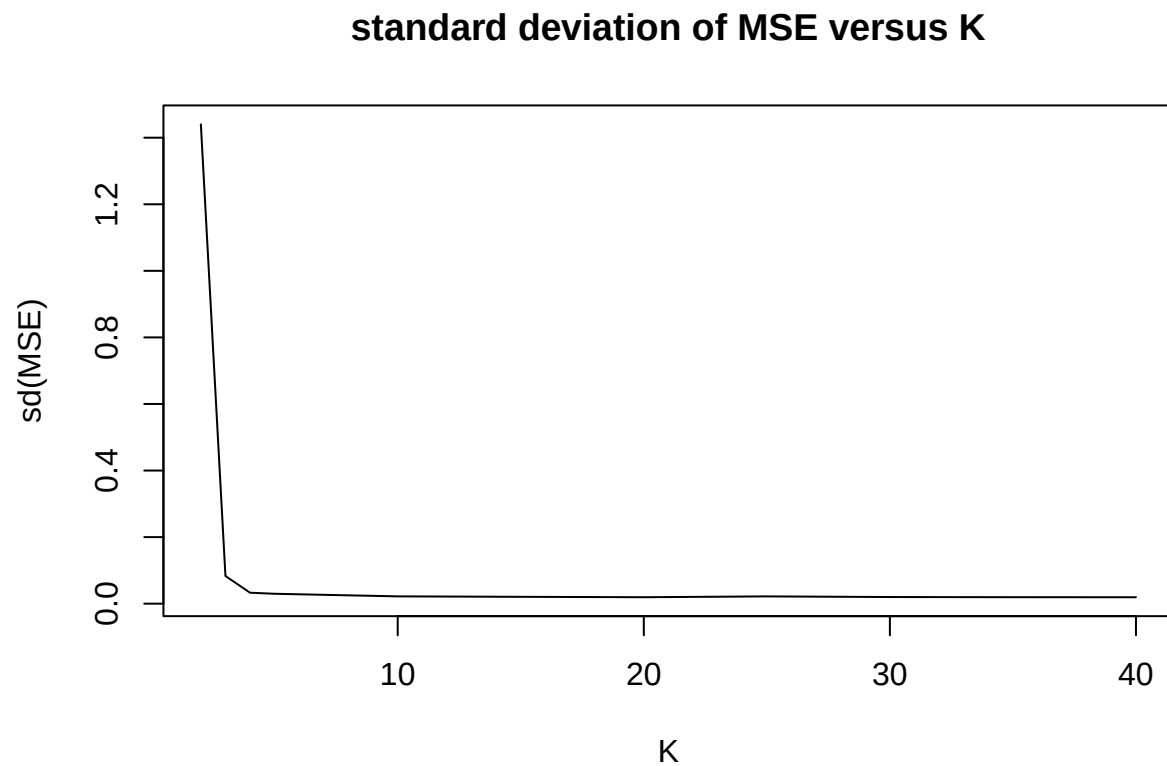
  meanvals40[j]=1-mean(cv.error,na.rm=T)
  sdvals40[j]=sd(cv.error,na.rm=T)
}
```

Note that the above simulation takes a while to run with $R = 1000$. So that I don't have to rerun the simulation every time that I run the R Markdown document, I saved the simulation output to an R image file and then I reload it to make the plot:

```
load("meanvals_sdvals.Rdata")
plot(Kvals40,meanvals40, xlab="K",ylab="1-mean(MSE)",main="1-MSE versus K", type="l")
```



```
plot(Kvals40,sdvals40,xlab="K", ylab="sd(MSE)", main="standard deviation of MSE versus K",type="l")
```



Here is the table:

```
dd1=data.frame(Kvals40,1-meanvals40,sdvals40)
names(dd1)=c("K", "mean MSE", "SD of MSE")
round(dd1,4)
```

##	K	mean MSE	SD of MSE
## 1	2	0.2027	1.4401
## 2	3	0.1154	0.0832
## 3	4	0.1064	0.0329
## 4	5	0.1031	0.0297
## 5	10	0.1007	0.0220
## 6	15	0.0996	0.0206
## 7	20	0.0989	0.0194
## 8	25	0.0993	0.0219
## 9	30	0.0983	0.0199
## 10	35	0.0973	0.0195
## 11	40	0.0995	0.0193

This output is similar to that at the link in that there is little variation in mean MSE or standard deviation in the MSE for all values of K except $K = 2 - 5$. For this example, at least, the value of K does not matter much provided that it is at least 5. I am getting somewhat higher standard deviations than the link and my standard deviation for $K=2$ is much higher. Bear in mind that $K = N$ is LOOCV. Thus, these results that there is no difference in performance between 10-fold CV and LOOCV for this scenario. While there is minimal difference in the mean MSW between 5- and 10-fold CV, the standard deviation of the MSE is substantially larger for $K=5$ than $K=10$. Based on this, I would recommend 10-fold CV over 5-fold.

Sample size 2000

We will repeat the simulation with a sample size of 200:

```
set.seed(12345)
R=1000
N=200
s=0.5
Kvals=c(2:4,seq(from=5,to=N,by=5))
meanvals200=vector(length=length(Kvals))
sdvals200=vector(length=length(Kvals))
j=0

for(K in Kvals)
{ j=j+1
  print(K)
  cv.error=vector(length=R)

  for(i in 1:R)
  { x=runif(N,min=0,max=2.5)
    y=sin(x)+runif(N,min=-s,max=s) #rnorm(N,sd=sigma0)
    dat=data.frame(x,y)
    glm.fit=glm(y~x+I(x^2)+I(x^3)+I(x^4),data=dat)
    cv.error[i]=cv.glm(dat,glm.fit,K=K)$delta[1]
  }
}
```



```

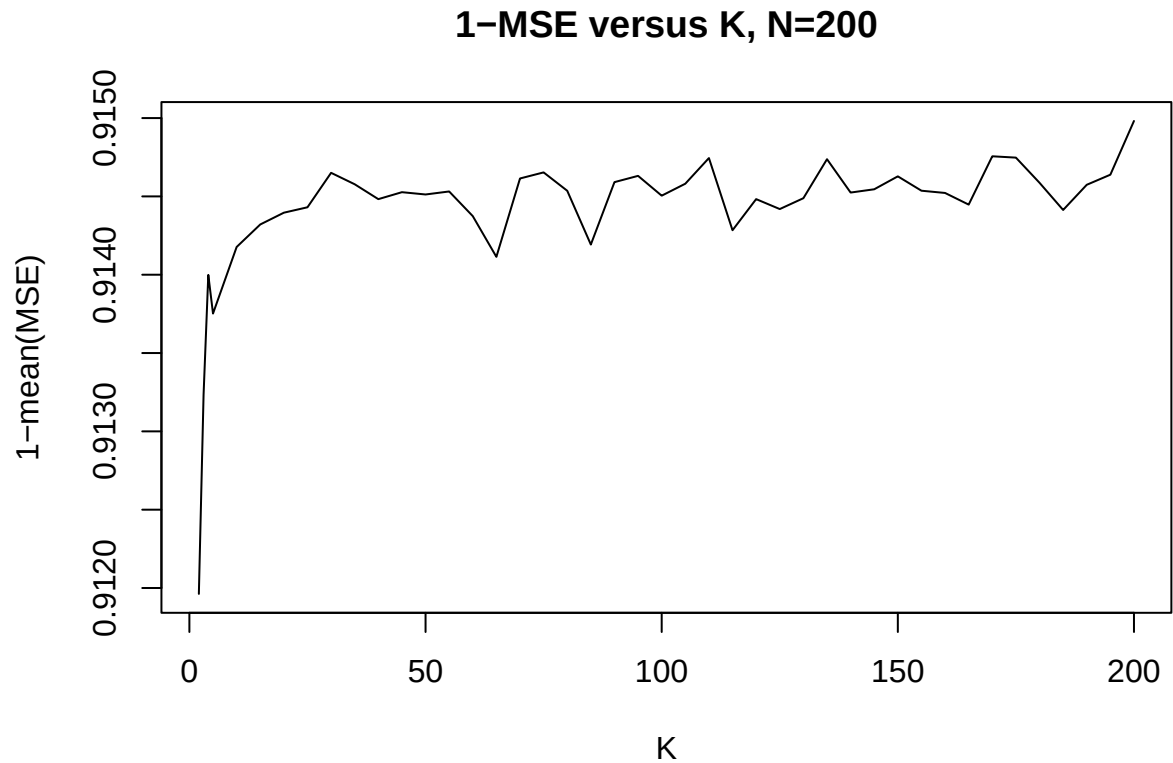
meanvals200[j]=1-mean(cv.error,na.rm=T)
sdvals200[j]=sd(cv.error,na.rm=T)
}

```

```

plot(Kvals200,meanvals200, xlab="K",ylab="1-mean(MSE)",main="1-MSE versus K, N=200", type="l")

```

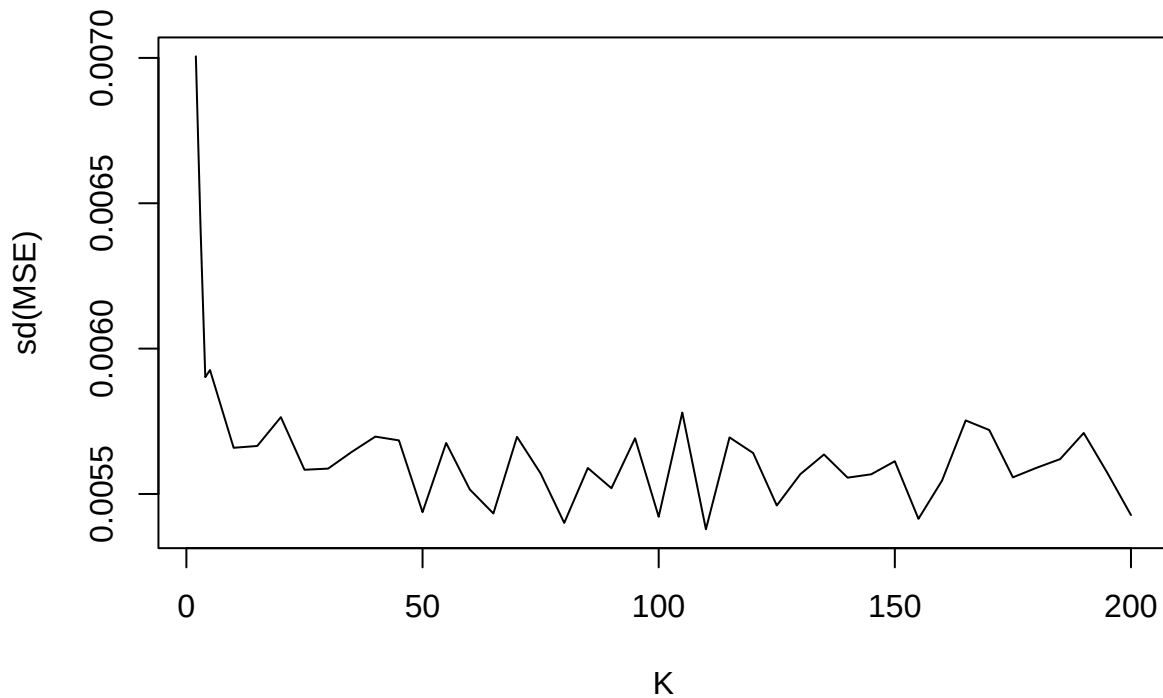


```

plot(Kvals200,sdvals200,xlab="K", ylab="sd(MSE)", main="standard deviation of MSE versus K, N=200",type="l")

```

standard deviation of MSE versus K, N=200



Here is the data frame:

```
dd2=data.frame(Kvals200,1-meanvals200,sdvals200)
names(dd2)=c("K","mean MSE","SD of MSE")
round(dd2[c(1:7,9,11,13,17,23,28,33,38,43),],4)
```

##	K	mean MSE	SD of MSE
## 1	2	0.0880	0.0070
## 2	3	0.0868	0.0064
## 3	4	0.0860	0.0059
## 4	5	0.0862	0.0059
## 5	10	0.0858	0.0057
## 6	15	0.0857	0.0057
## 7	20	0.0856	0.0058
## 9	30	0.0853	0.0056
## 11	40	0.0855	0.0057
## 13	50	0.0855	0.0054
## 17	70	0.0854	0.0057
## 23	100	0.0855	0.0054
## 28	125	0.0856	0.0055
## 33	150	0.0854	0.0056
## 38	175	0.0853	0.0056
## 43	200	0.0850	0.0054

With a sample size of 200, we see less variation in the mean and standard deviation of prediction error than with a sample size of 40. We can see a small change going from $K = 2 - 5$ but the difference seems negligible.

This again supports the finding that the value of K in K-fold cross-validation does not have much impact on the results.

This code chunk saves the simulation results for future use:

```
#save(Kvals40,Kvals200,meanvals40,meanvals200,sdvals40,sdvals200,file="meanvals_sdvals.Rdata")
```

Problem 3: Bootstrap Confidence Intervals

In order to study the coverage properties of the confidence intervals, we will conduct a simulation study. The basic structure of the simulation is

loop r over replicates

- ```
{
1. generate data set
2. calculate CI for data set
3. check whether CI includes true parameter value
}
```

CI coverage =  $\frac{\text{number of replicate with CIs covering true value}}{\text{number of simulation replicates}}$

First, I will write a function that conducts the above procedure. This function takes input of 1) sample size  $N$ , 2) number of simulation replicates  $nrep$ , 3) number of bootstrap replicates  $R$ , and 4) the test statistic `bootstat`. It returns the percent coverage for confidence intervals calculated by percentile bootstrap, adjusted bootstrap, and the normal approximation.

```
bootsim<-function(N,nrep,R,bootstat)
{#input: N=sample size, nrep=# of simulation replicates, R=number of bootstrap reps
#output: vector: (CI coverage for bootstrap percentile, adjusted bootstrap CI, normal CI)

BIp<-matrix(nrow=nrep,ncol=2) #matrix to contain CIs for percentile CI
BIa<-matrix(nrow=nrep,ncol=2) #matrix to contain CIs for adjusted CI
BIb<-matrix(nrow=nrep,ncol=2)
NI<-matrix(nrow=nrep,ncol=2) #matrix to contain CIs for normal CI
tp<-0 #true parameter value

if(R>=100){
 for(i in 1:nrep) #loop over replicates
 { dat1<-rnorm(N,mean=tp) #generate data
 #dat1<-rexp(N,tp)
 bout<-boot.ci(boot(dat1,bootstat,R),type=c("norm", "perc", "bca","basic")) # conduct bootstrap

 BIp[i,<=2]<-bout$percent[4:5] #percentile interval
 BIa[i,<=2]<-bout$bca[4:5] #adjusted bootstrap interval
 BIb[i,<=2]<-bout$basic[4:5] #basic interval
 NI[i,<=2]<-bout$norm[,2:3]
 }

 #find coverage for each CI:
 coverage<-c(sum((BIp[,1]<tp)*(BIp[,2]>tp))/nrep,sum((BIa[,1]<tp)*(BIa[,2]>tp))/nrep,sum((BIb[,1]<tp)*(BIb[,2]>tp))/nrep)

} else{ #adjusted bootstrap doesn't work for R<100

 for(i in 1:nrep) #loop over replicates
 { dat1<-rnorm(N,mean=tp) #generate data
 #dat1<-rexp(N,tp)
```

```

 bout<-boot.ci(boot(dat1,bootstat,R),type=c("norm", "perc","basic")) # conduct bootstrap
 BIp[i,<-bout$percent[4:5] #percentile interval
 BIb[i,<-bout$basic[4:5] #percentile interval
 NI[i,<-bout$norm[,2:3]
 }

 #find coverage for each CI:
 coverage<-c(sum((BIp[,1]<tp)*(BIp[,2]>tp))/nrep,NA,sum((BIb[,1]<tp)*(BIb[,2]>tp))/nrep,sum((NI[,1]
}

#return(paste("Percentile:",coverage[1]," Adjusted: ",coverage[2]," Normal approximation:",coverage[3]))
return(coverage)

}

```

## Sample mean as estimator

First, we will use the sample mean as the estimator for the population mean of the normal distribution.

Thus, I make a wrapper function for the mean to be used in the `boot` function. I will also go ahead and make a similar function for the median.

```

library(boot)
nrep=1000 #number of simulation replicates

bootmean<-function(x,i) mean(x[i]) # function calc bootstrap mean

bootmedian<-function(x,i) median(x[i]) # function calc bootstrap median

```

Now, I run the simulations for five cases of sample size, using the sample mean as the test statistic:

Note: The simulations for this homework take substantial time to run. Thus, knitting the R Markdown file takes a long time if we rerun all of the simulations every time. To avoid this, I only run each simulation one time and save the results using the `save` function. In the example below, I save the simulation output `a1`, `a2`, `a3`, `a4`, and `a5` into a file called `bootsima.RData`. I use the `eval=FALSE` option in the chunk header so that this chunk does not run when I knit the pdf (you can see this in the `.rmd` file, but not the pdf). I then load the saved file using the `load` function. This loads the saved objects `a1`, `a2`, `a3`, `a4`, and `a5` into memory so that I can use those for plots. I normally use the option `echo=FALSE` in the chunk that loads these objects. This makes it so that these code chunks don't show in the knitted pdf file. I don't do that here since I want you to see it.

```

Nvec<-c(5,10,30,50,100)

a1=bootsim(Nvec[1],nrep,1000,bootmean) #sample size=5
a2=bootsim(Nvec[2],nrep,1000,bootmean) #sample size=10
a3=bootsim(Nvec[3],nrep,1000,bootmean) #sample size=30
a4=bootsim(Nvec[4],nrep,1000,bootmean) #sample size=100
a5=bootsim(Nvec[5],nrep,1000,bootmean) #sample size=100

```

```
save(a1,a2,a3,a4,a5,file="bootsima.RData")
```

```
load("bootsima.RData")
#I saved the simulation results so that I don't have to rerun every time
```

Put the simulation results into a table:

```
library(knitr)
Nvec<-c(5,10,30,50,100)
CItab1=data.frame(Nvec,rbind(a1,a2,a3,a4,a5),row.names=NULL)
kable(CItab1,col.names=c("N","Percentile","Adjusted","Basic","Normal Approx"))
```

| N   | Percentile | Adjusted | Basic | Normal Approx |
|-----|------------|----------|-------|---------------|
| 5   | 0.845      | 0.844    | 0.839 | 0.849         |
| 10  | 0.895      | 0.892    | 0.895 | 0.892         |
| 30  | 0.938      | 0.936    | 0.944 | 0.937         |
| 50  | 0.947      | 0.946    | 0.950 | 0.945         |
| 100 | 0.947      | 0.947    | 0.946 | 0.947         |

The above table shows the simulated coverage of the confidence intervals. Ideally, the actual coverage should equal the nominal coverage. Thus, if the CIs are being calculated correctly, we should see values near 0.95. All of the CIs perform poorly with low sample size. Note that the **normal** CI is calculated assuming that the variance is known (i.e. using the Z rather than the  $t$ ). We saw in the previous assignment that the normal CI works well for low sample size if we use the  $t$  to account for the estimated variance.

As the sample sizes increase to 30 and above, all four methods perform reasonably well. At sample sizes of 30, 50, and 100 the coverage is slightly under the target at about 94%.

### Effect of number of bootstrap replicates:

Next, we will see how the number of bootstrap replicates affects performance. I will take a sample size of 100, since we saw in the previous section that we hit approximately the target coverage with a sample size of 100 and 1000 bootstrap replicates. We will now try smaller numbers of bootstrap replicates:

```
b1=bootsim(100,nrep,30,bootmean)
#CI coverage=0.919, 0.881, 0.941 for percentile, adjusted, normal
b2=bootsim(100,nrep,50,bootmean)
#CI coverage=0.945, 0.928, 0.942 for percentile, adjusted, normal
b3=bootsim(100,nrep,100,bootmean)
#CI coverage=0.938, 0.925, 0.945 for percentile, adjusted, normal
b4=bootsim(100,nrep,500,bootmean)
#CI coverage=0.938, 0.925, 0.945 for percentile, adjusted, normal

save(b1,b2,b3,b4,file="bootsimb.RData")
```

```
load(file="bootsimb.RData")

CItab2=data.frame(c(30,50,100,500),rbind(b1,b2,b3,b4),row.names=NULL)

kable(CItab2,col.names=c("R","Percentile","Adjusted","Basic","Normal Approx"))
```

| R   | Percentile | Adjusted | Basic | Normal | Approx |
|-----|------------|----------|-------|--------|--------|
| 30  | 0.936      | NA       | 0.934 |        | 0.931  |
| 50  | 0.959      | NA       | 0.959 |        | 0.949  |
| 100 | 0.943      | 0.939    | 0.933 |        | 0.927  |
| 500 | 0.939      | 0.942    | 0.944 |        | 0.942  |

The adjusted CI doesn't work for R less than 100.

Other than this, the number of bootstrap replicates doesn't have a large effect. All three bootstrap methods have at least 94% coverage for R=30 and above and reach approximately 95% coverage at N=50 and above.

The normal approximation CI is at about 94% coverage at N=30 and 50 and achieves approximately 95% coverage at N=100 and 500.

Thus, in summary, we see that for sample sizes of approximately 30 and higher and if we have at least 100 bootstrap replicates, then we will get reasonable performance from all methods of calculating CIs. Since the CI based on the normal approximation is easier to calculate, it is superior for the case for which is was intended - the mean of a normal distribution.

## Median

Next, we repeat the above simulations but using the sample median as the estimator.

```

c1=bootsim(Nvec[1],nrep,1000,bootmedian) #sample size=5
c2=bootsim(Nvec[2],nrep,1000,bootmedian) #sample size=10
c3=bootsim(Nvec[3],nrep,1000,bootmedian) #sample size=30
c4=bootsim(Nvec[4],nrep,1000,bootmedian) #sample size=100
c5=bootsim(Nvec[5],nrep,1000,bootmedian) #sample size=100

save(c1,c2,c3,c4,c5,file="bootsimc.RData")

load(file="bootsimc.RData")

CItab1=data.frame(Nvec,rbind(c1,c2,c3,c4,c5),row.names=NULL)

kable(CItab1,col.names=c("N","Percentile","Adjusted","Basic","Normal Approx"))

```

| N   | Percentile | Adjusted | Basic | Normal | Approx |
|-----|------------|----------|-------|--------|--------|
| 5   | 0.928      | 0.777    | 0.780 |        | 0.892  |
| 10  | 0.936      | 0.931    | 0.823 |        | 0.896  |
| 30  | 0.941      | 0.941    | 0.847 |        | 0.909  |
| 50  | 0.956      | 0.953    | 0.885 |        | 0.930  |
| 100 | 0.949      | 0.949    | 0.883 |        | 0.925  |

We see that all of the CI's except percentile do poorly for N=5. The Adjusted and Basic CI's have less than 80% coverage, while the normal CI has about 91% coverage.

The Percentile CI achieves 94-95% coverage at all sample sizes. The Adjusted bootstrap CI does less well. It has coverage of 79% at N=5, 92% at N=10, and reaches 94-95% coverage for sample sizes of 50 and 100. The Basic CI does poorly at all sample sizes, never getting above 88% coverage. The Normal Approximation does poorly at all sample sizes, but improves with sample size.

In summary, we see that the Percentile CI performs well in all circumstances when the median is the test statistic. Not surprisingly, the normal approximation does not perform well. This is expected because the median of the normal does not have normal distribution. It is surprising that the Adjusted Bootstrap CI does worse than the Percentile, because it is supposed to be a superior method.

## Problem 4

In this problem, I will use simulations to explore how the training error and prediction error vary with respect to the complexity of the model fit and the residual error.

In class, we defined the **optimism** of the training error:

$$op = Err_{in} - \overline{err}$$

where  $Err_{in}$  is the in-sample prediction error and  $\overline{err}$  is the training error. This is the amount by which the training error underestimates the prediction error when the trained model is applied to new data.

We also defined the average optimism as the expectation over training data sets:

$$\omega = E_{\tau}(op)$$

Our goal will be to find see how the average optimism varies with model complexity and amount of residual error.

## Data generating functions

I will start by making some functions for generating data.

Function to make data:

```
makedata<-function(n,sd0,x1=0,x2=10,x=NULL, f)
{
 if(is.null(x)) x=sort(runif(n,x1,x2))

 y=f(x)+rnorm(n,sd=sd0)

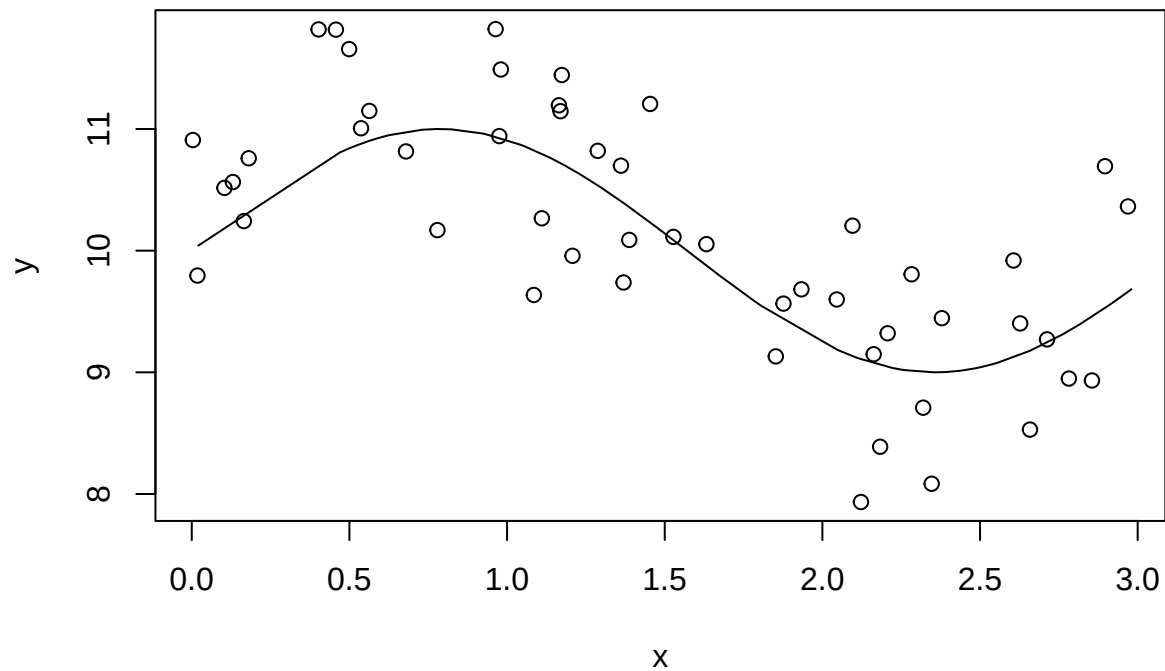
 return(data.frame('x'=x,'y'=y))
}
```

Function for the underlying relationship between  $X$  and  $Y$ . This function is  $f(x) = 10 + \sin(2x)$ .

```
f2<-function(x)
{
 return(10+sin(2*x))
}

par(mfrow=c(1,1))
```

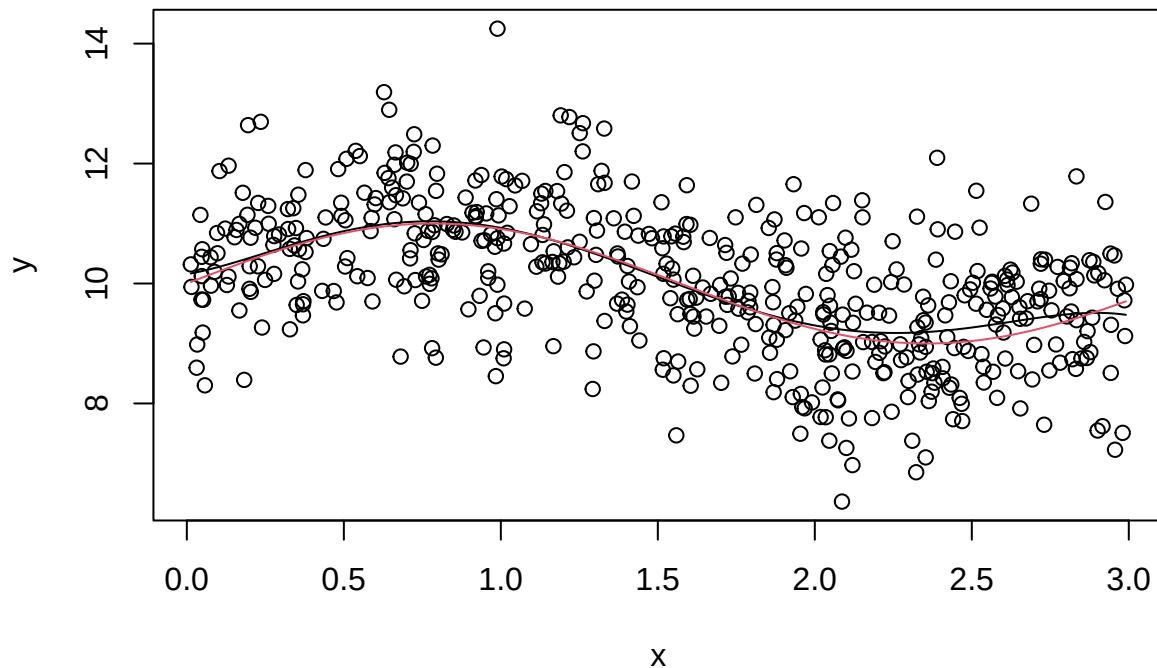
```
plot(makedata(n=50,sd0=0.5,x2=3,f=f2))
lines(makedata(n=50,sd0=0,x2=3,f=f2))
```



This code can be used to plot a fitted model with the data.

```
ntrain=500
par(mfrow=c(1,1))
xmax=3
xdat=sort(runif(ntrain,0,xmax))
traindat=makedata(n=ntrain,sd0=1,x=xdat,f=f2)
trainmod=lm(y~poly(x,5),data = traindat)
plot(traindat)
lines(traindat$x,predict(trainmod))
lines(traindat$x,f2(traindat$x),col=2)
```





### In-Sample Prediction error

In order to find the in-sample prediction error, we keep the set of x-values fixed. I create a vector `xdat` of x values that will be used throughout the simulations.

The code below finds the prediction error for a fixed training data set. The basic structure of the program is

```
generate fixed x values
generate training data y values
find training error
```

```
loop r over simulation replicates { generate test data find predicted values for test data from
trained model
find and store MSE of predicted vs actual for test data }
```

```
optimism=training error-mean(prediction MSE)
```

The comments explain the code. A polynomial model of degree 8 is fit with the training data using `lm(y~poly(x,d),data = traindat)`. This is then used to predict the test data.

```
R=1000 #number of sim reps for finding prediction error
d=8 #degree of polynomial to be fit
sd1=0.5 #SD of residuals
ntrain=500 #training sample size
ntest=500 #test sample size
xmax=3 #max extent of x values
fx=f2 #model to use

predErr=vector(length=R) #vector to store MSE of prediction error across replicates

xdat=sort(runif(ntrain,0,xmax))

#make training data set, fit model, find training error
```

```
traindat=makedata(n=ntrain,sd0=sd1,x=xdat,f=fx)
trainmod=lm(y~poly(x,d),data = traindat)
(trainErr=mean((traindat$y-predict(trainmod))^2))
```

```
[1] 0.2557052
```

```
#loop over sim reps
for(r in 1:R)
{ #make training data set, make predictions from trained model, find pred error
 testdat=makedata(n=ntest,sd0=sd1,x=xdat,f=fx)
 predY=predict(trainmod,newdata=testdat)
 predErr[r]=mean((testdat$y-predY)^2)
}

mean(predErr)
```

```
[1] 0.255433
```

```
trainErr
```

```
[1] 0.2557052
```

```
(opt=mean(predErr)-trainErr)
```

```
[1] -0.0002721427
```

## Average Optimism

The above is finding the optimism for a fixed training data set.

Next, we remove the conditioning on the training data set by doing adding variability in the training data. We add a second simulation replicate loop. Here is an outline of the code:

**generate fixed x values**

**loop r1 over training data sim reps { generate training data y values  
find and store training error**

```
loop r2 over test data simulation replicates
{ generate test data
 find predicted values for test data from trained model
 find and store MSE of predicted vs actual for test data
}
```

```
}
```

```
**avg optimism=mean(training error)-mean(prediction MSE)
```

The code is explained in the comments:

```

R1=100 #number of training data reps
R2=100 #number of test data reps
d=5 #degree of polynomial fit
sd1=1 #SD of residuals
ntrain=500 #training data sample size
ntest=ntrain #test data sample size
fx=f2 #model to use

xmax=3
xdat=sort(runif(ntrain,0,xmax))

trainErr=vector(length = R1)
meanPredErr=vector(length=R1)

for (r1 in 1:R1)
{
 #print(r1)
 traindat=makedata(n=ntrain,sd0=sd1,x=xdat,f=fx)
 trainmod=lm(y~poly(x,d, raw=TRUE),data = traindat)
 trainErr[r1]=mean((traindat$y-predict(trainmod))^2)

 predErr=vector(length=R2)
 for(r2 in 1:R2)
 {
 testdat=makedata(n=ntest,sd0=sd1,x=xdat,f=fx)
 predY=predict(trainmod,newdata=testdat)
 predErr[r2]=mean((testdat$y-predY)^2)
 }

 meanPredErr[r1]=mean(predErr)
 #optvec[r1]=mean(predErr)-trainErr
}

mean(meanPredErr)

```

```
[1] 1.012999
```

```
mean(trainErr)
```

```
[1] 0.9916718
```

```
mean(meanPredErr)-mean(trainErr)
```

```
[1] 0.0213271
```

```
#mean(meanPredErr)
```

Next, we turn this into a function:

```

#function to simulate training error, test error, optimsim
#input:
#R1,R2=number of sim reps for finding prediction error
d=degree of polynomial to be fit
sd1=SD of residuals
ntrain=training sample size
ntest=test sample size
xmax=max extent of x values
fx=model to use

sim_Error<-function(R1,R2,sd1,ntrain,ntest,xmax,d,fx)
{
 xdat=sort(runif(ntrain,0,xmax))

 trainErr=vector(length = R1)
 meanPredErr=vector(length=R1)

 for (r1 in 1:R1)
 {
 #if(r1/10==floor(r1/10)) print(r1)
 traindat=makedata(n=ntrain,sd0=sd1,x=xdat,f=fx)
 trainmod=lm(y~poly(x,d, raw=TRUE),data = traindat)
 trainErr[r1]=mean((traindat$y-predict(trainmod))^2)

 predErr=vector(length=R2)
 for(r2 in 1:R2)
 { testdat=makedata(n=ntest,sd0=sd1,x=xdat,f=fx)
 predY=predict(trainmod,newdata=testdat)
 predErr[r2]=mean((testdat$y-predY)^2)
 } #end r2 loop

 meanPredErr[r1]=mean(predErr)
 #optvec[r1]=mean(predErr)-trainErr
 } #end r1 loop

 outlist=list(`pred err`=mean(meanPredErr),`train err`=mean(trainErr),`opt`=mean(meanPredErr)-mean(trainErr))
 return(outlist)
}

```

Test the function:

```
sim1=sim_Error(R1=500,R2=500,sd=1,d=3,xmax=3,ntrain=50,ntest=50,fx=f2)
```

## Simulations

The goal is to see how the average optimism varies with the model complexity and the magnitude of the residual error.

I will run simulations of the average optimism over values of the degree of the polynomial from 1 to 10. I do this for three values of the standard error of the residuals, 0.5, 1, and 2.

```

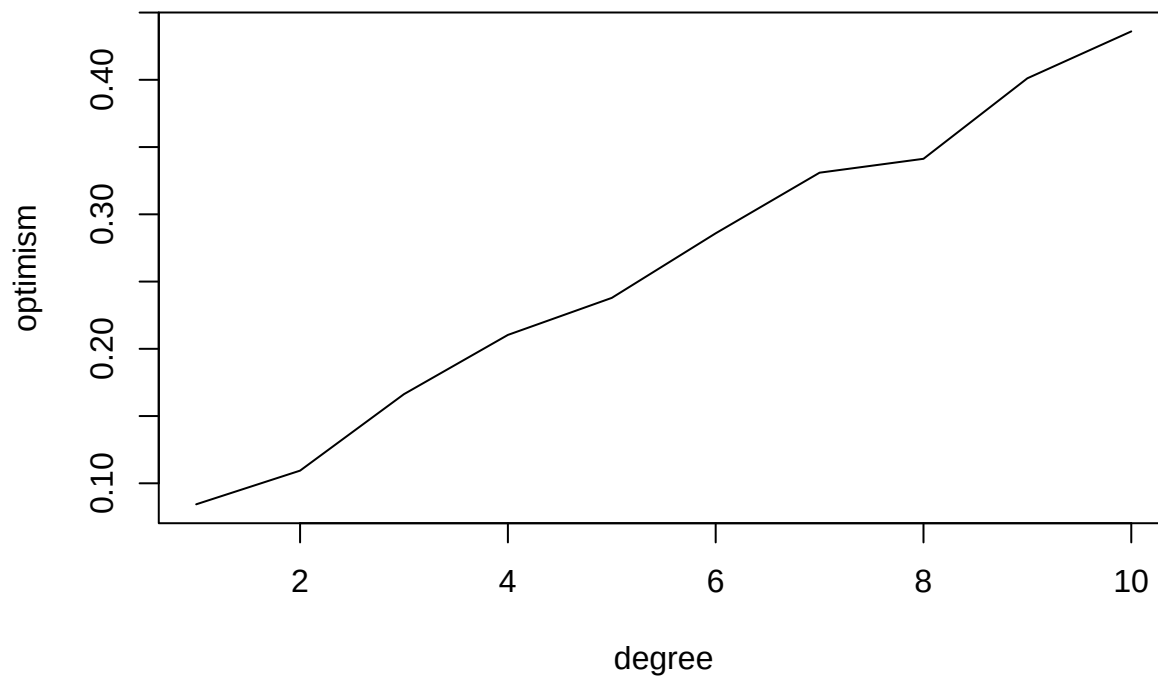
simout1=matrix(nrow=10,ncol=3)
for(dval in 1:10)
{ print(dval)
 simoutrun=sim_Error(R1=500,R2=500,sd=1,d=dval,xmax=3,ntrain=50,ntest=50,fx=f2)
 simout1[dval,1]=simoutrun$`pred err`
 simout1[dval,2]=simoutrun$`train err`
 simout1[dval,3]=simoutrun$`opt`
}

```

```

plot(1:10,simout1[,3],xlab="degree",ylab="optimism", type="l")

```



```

simout2=matrix(nrow=10,ncol=3)

for(dval in 1:10)
{ print(dval)
 simoutrun=sim_Error(R1=500,R2=500,sd=0.5,d=dval,xmax=3,ntrain=50,ntest=50,fx=f2)
 simout2[dval,1]=simoutrun$`pred err`
 simout2[dval,2]=simoutrun$`train err`
 simout2[dval,3]=simoutrun$`opt`
}

```

```

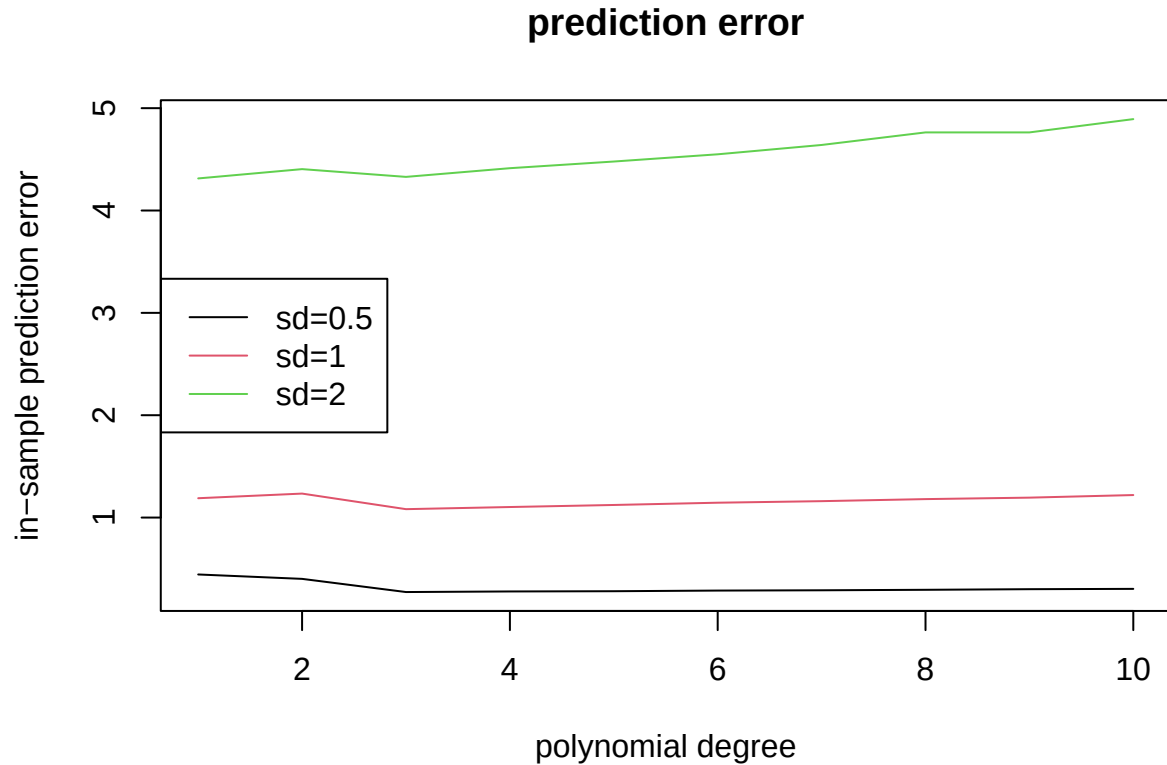
simout3=matrix(nrow=10,ncol=3)

for(dval in 1:10)
{ print(dval)
 simoutrun=sim_Error(R1=500,R2=500,sd=2,d=dval,xmax=3,ntrain=50,ntest=50,fx=f2)
 simout3[dval,1]=simoutrun$`pred err`
 simout3[dval,2]=simoutrun$`train err`
 simout3[dval,3]=simoutrun$`opt`
}

```

Plot the average in-sample error:

```
matplot(1:10,cbind(simout2[,1],simout1[,1],simout3[,1]),xlab="polynomial degree",ylab="in-sample predic
legend("left",c("sd=0.5","sd=1","sd=2"),lty=1,col=1:3)
```

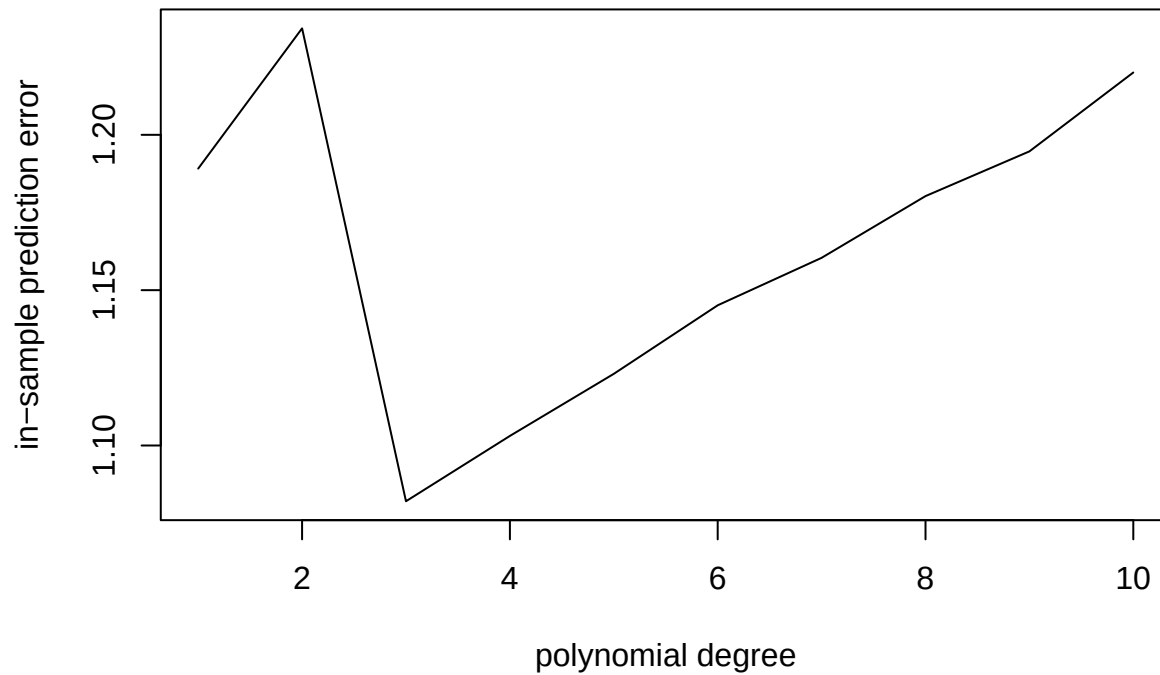


The in-sample prediction error increases as the residual error increases. This is expected - the higher residual variation makes prediction harder and thus the model is less accurate.

The plot below shows the prediction error for the case of standard deviation=0.5. I show this case on its own because it is hard to see what is happening when it is plotted with the other two curves.

```
plot(1:10,simout1[,1],type="l",xlab="polynomial degree",ylab="in-sample prediction error", main="predic
```

### prediction error for SD=0.5



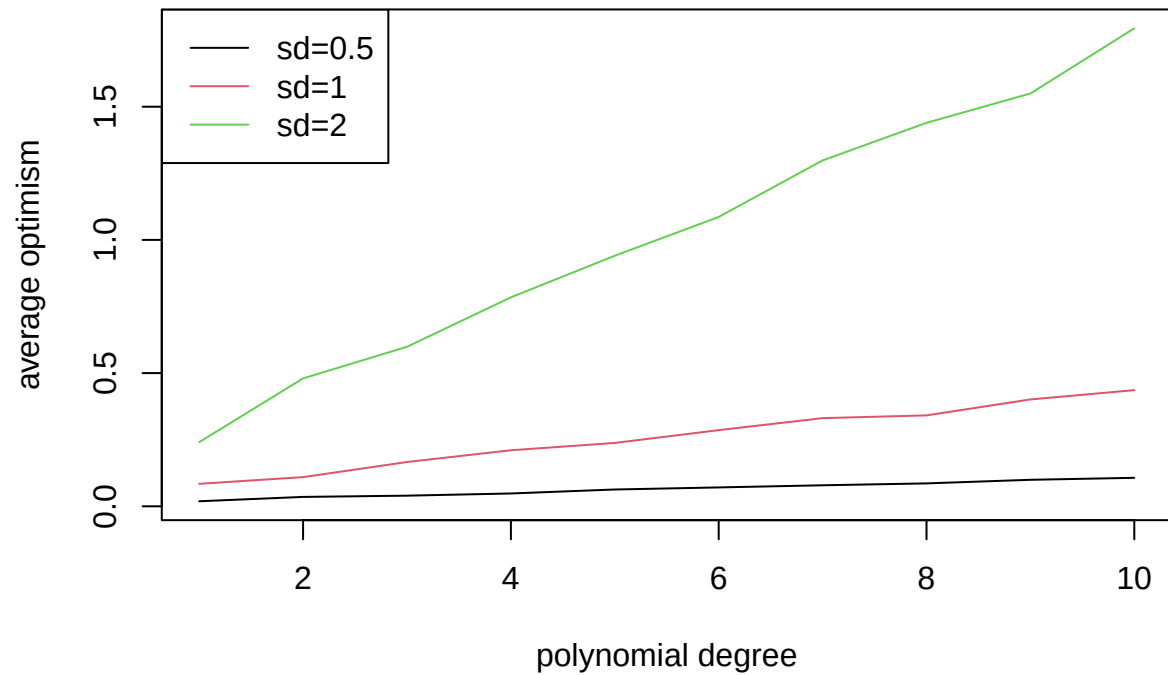
In this case, we see that the prediction error initially decreases with the model complexity. It reaches a minimum at degree 3 and then increases. This tells us that degree 3 is sufficient to fit the data and further model complexity results in overfitting.

For the case of standard deviation of 2, the prediction error is at its minimum for the degree 1-3 models and then increases with model complexity.

Plot the average optimism:

```
par(mfrow=c(1,1))
matplot(1:10,cbind(simout2[,3],simout1[,3],simout3[,3]),xlab="polynomial degree",ylab="average optimism",lty=1,col=1:3)
legend("topleft",c("sd=0.5","sd=1","sd=2"),lty=1,col=1:3)
```

## training optimism



We see that training optimism increases with model complexity and with the magnitude of residual error. The more complex the model, the more likely it is that the model is overfitting and it is overfitting that leads to training optimism. Of course, the model does a better job of predicting future data when the residual variance is lower.